

M2 — What this block gets you

- ✓ Replication vs migration— backfill once, then a live change stream; Datastream does both as one operation
- ✓ Change data capture (CDC) as a managed service— every INSERT, UPDATE, DELETE lands in BigQuery
- ✓ SQL Server is a supported Datastream source— your SQL Server → BigQuery path is first-class
- ✓ Log-based capture— no triggers, no polling, no production load (the answer for your M&O team)
- ✓ Hands-on: start a Datastream stream before lunch, validate the replication after it

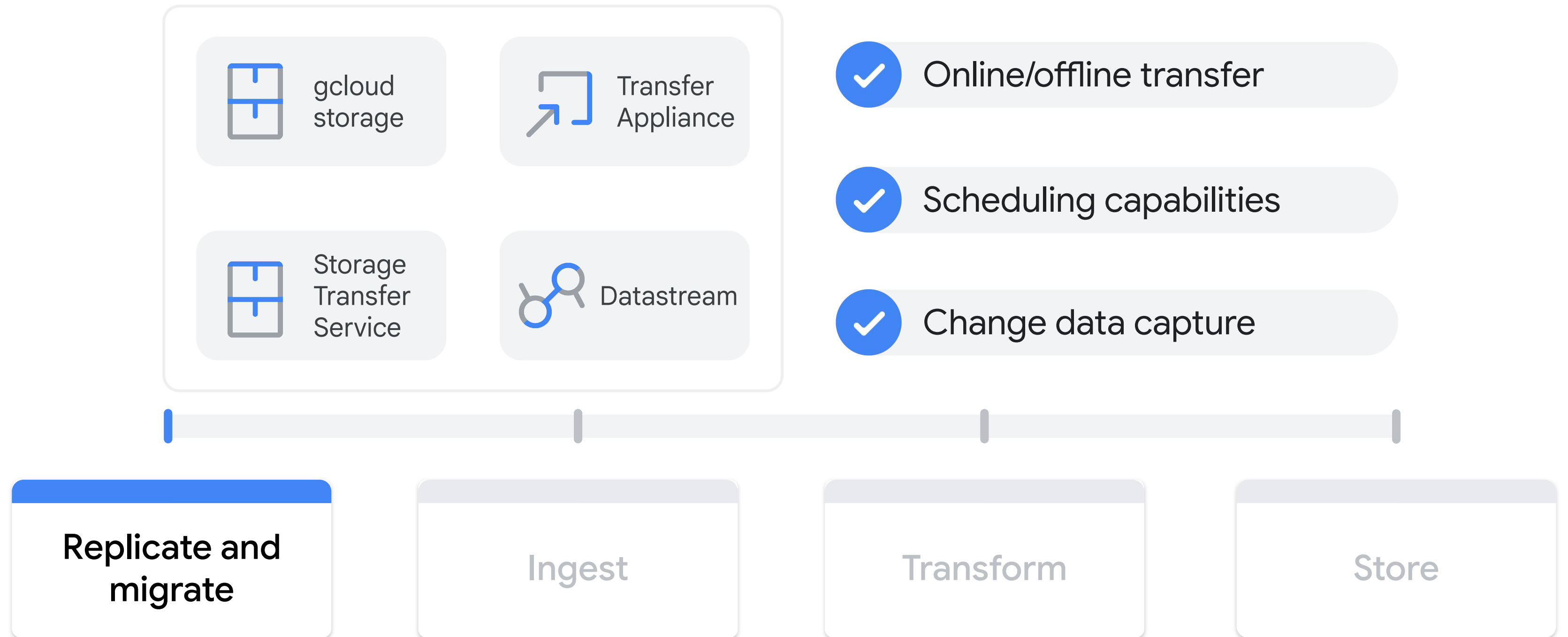
This block, across the lunch break

- | | | |
|-------------|---|-----|
| 11:10–11:50 | M2 — Data replication & migration: killing the nightly extracts | |
| 11:50–12:00 | Lab: Datastream PostgreSQL → BigQuery, part 1 — set up and start the stream | LAB |
| 12:00–13:00 | Lunch — the stream replicates while we eat | |
| 13:00–13:15 | Lab part 2 + CDC demo — validate backfill, watch a live change land | LAB |
| 13:15–13:45 | M3 — Extract & Load: the pattern for data that still arrives as files | |

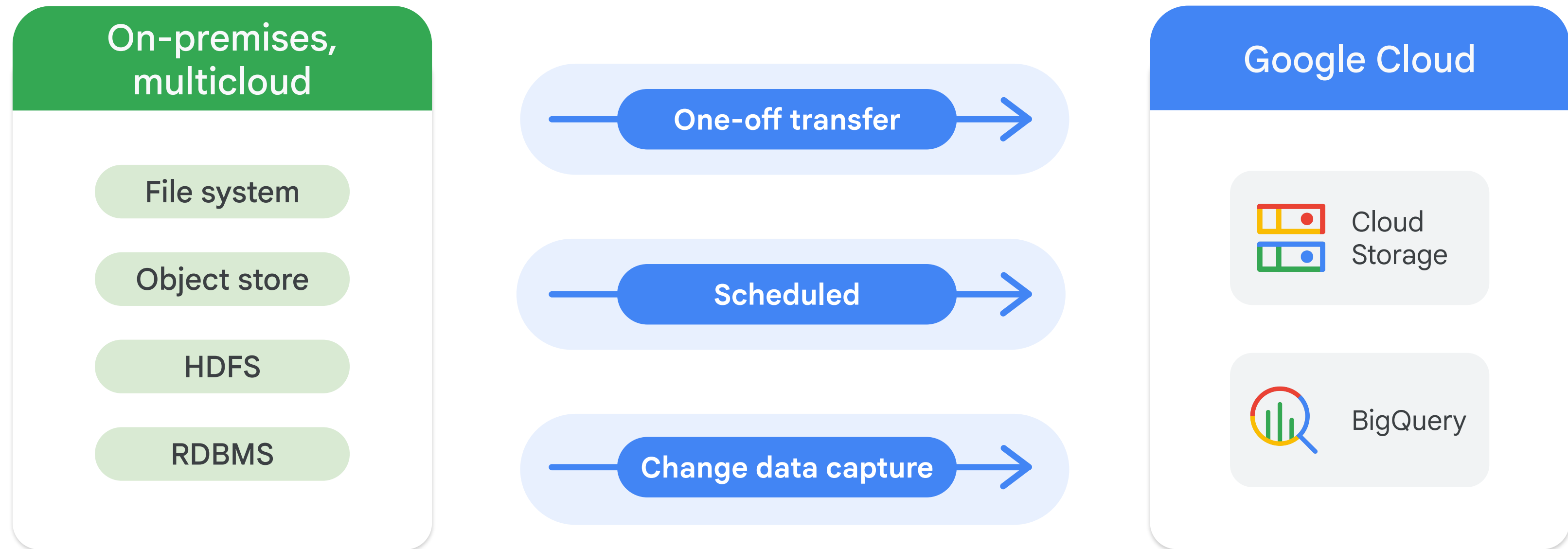


Data replication and migration

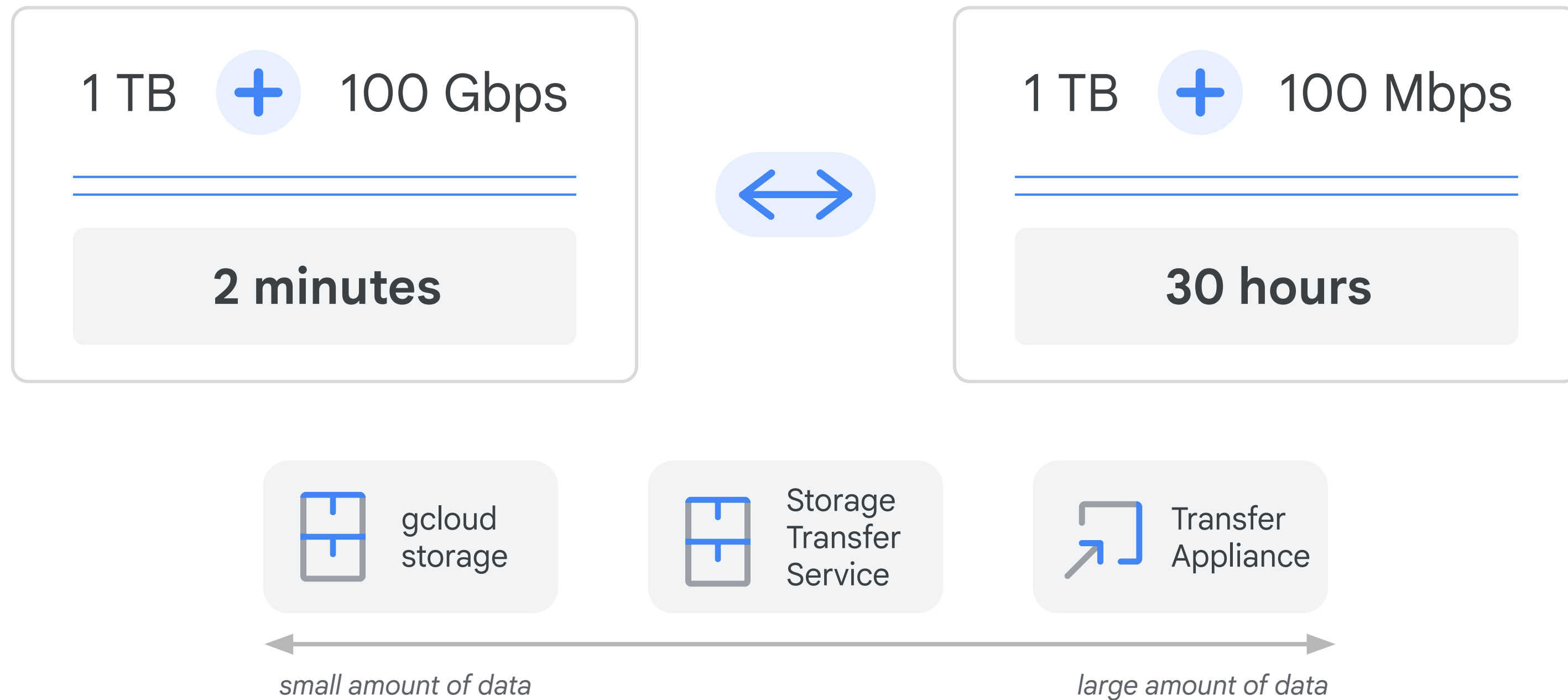
Replication and migration services onboard your data into Google Cloud



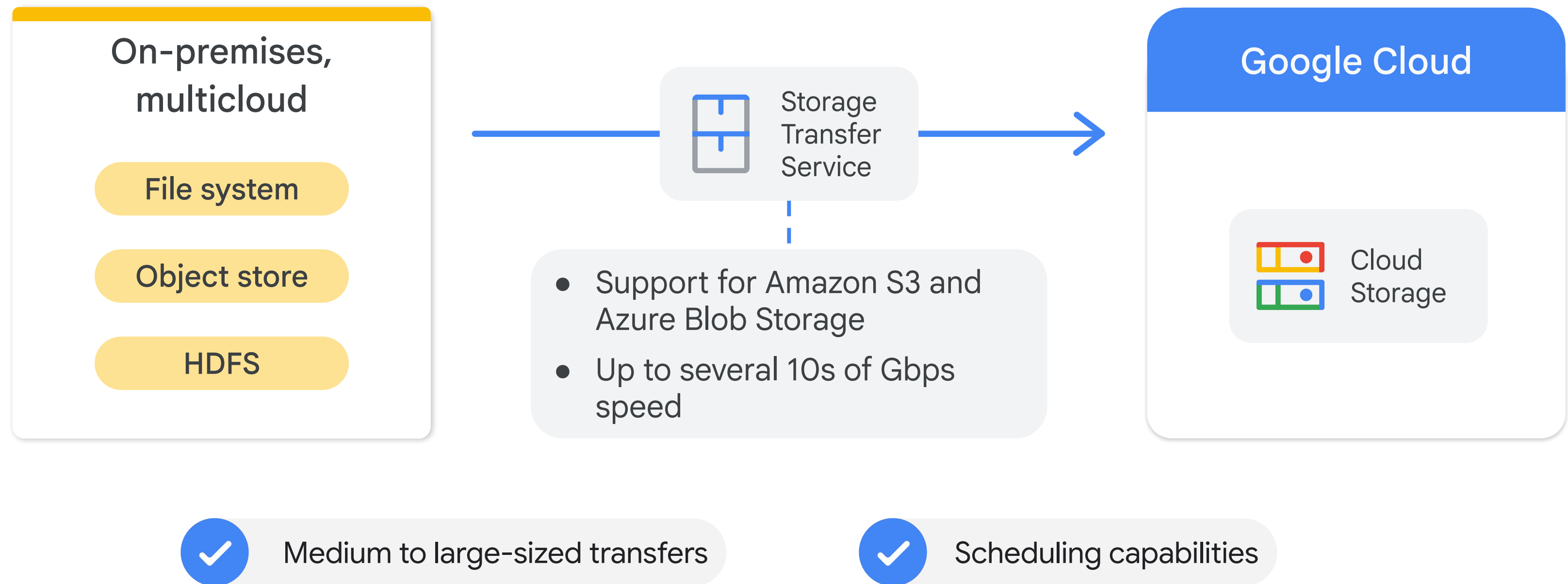
Common replication and migration scenarios for ingesting your data into Google Cloud



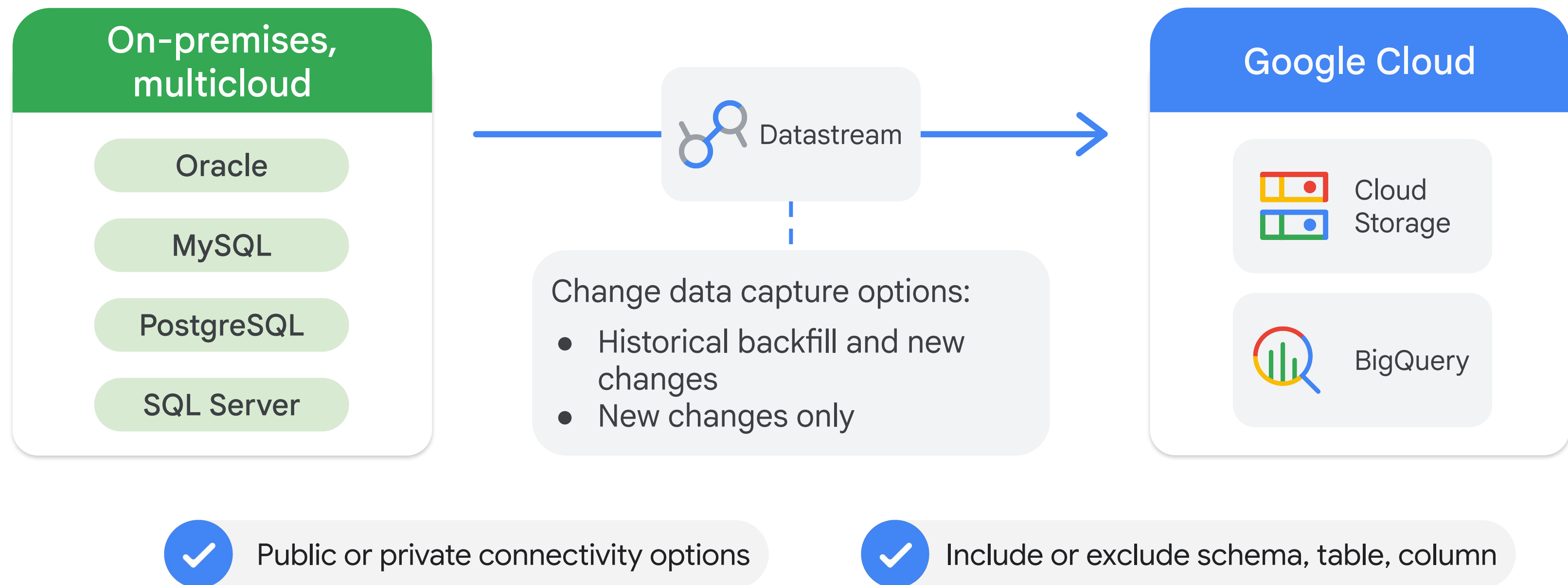
Choosing between migration options depends on your amount of data and network bandwidth



Move large amounts of data with Storage Transfer Service



Datastream continuously replicates your RDBMS into Google Cloud for analytics



SQL Server IS a supported Datastream source

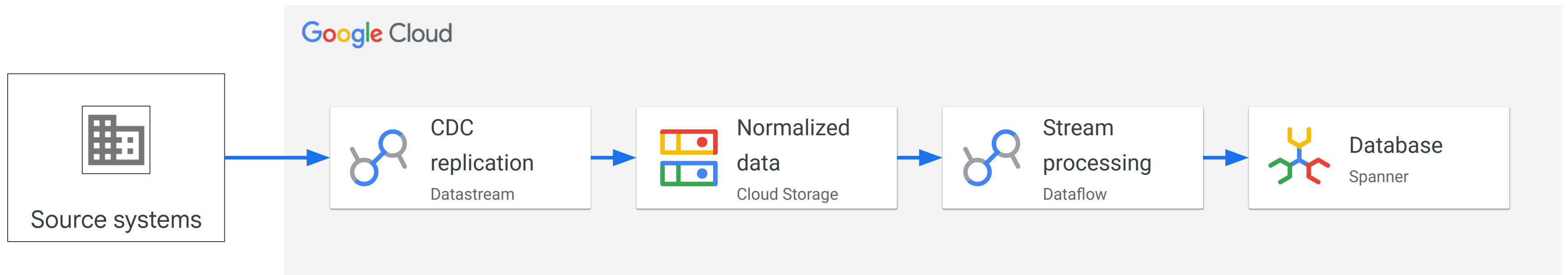
The deck's source graphic is the short list — SQL Server is fully supported.

- Self-managed or cloud-hosted SQL Server— supported alongside Oracle, MySQL and PostgreSQL
- Captured from the SQL Server transaction logs— the same log your databases already write
- Your SQL Server → BigQuery replication is a first-class Datastream scenario, not a workaround
- Today's lab uses PostgreSQL only because it is free-tier friendly— the mechanics are identical
- This lab is your project's pilot: source database → continuous changes → BigQuery

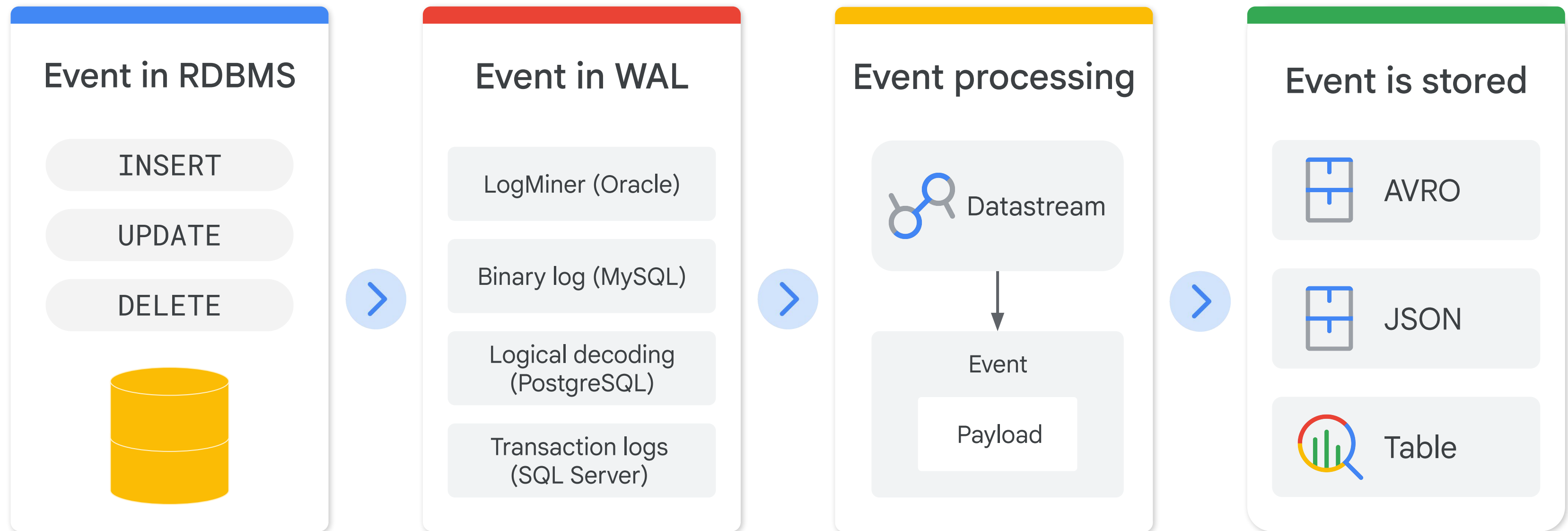
If the room takes one sentence from this module: Datastream supports SQL Server, and that is your path.

Datastream use cases

- Analytics with database replication into BigQuery
- Analytics with custom data processing in Dataflow into BigQuery
- Data processing using an event-driven architecture
- **Example:** database replication and migration using Dataflow templates



Datastream uses the database's write-ahead log (WAL) to process change events



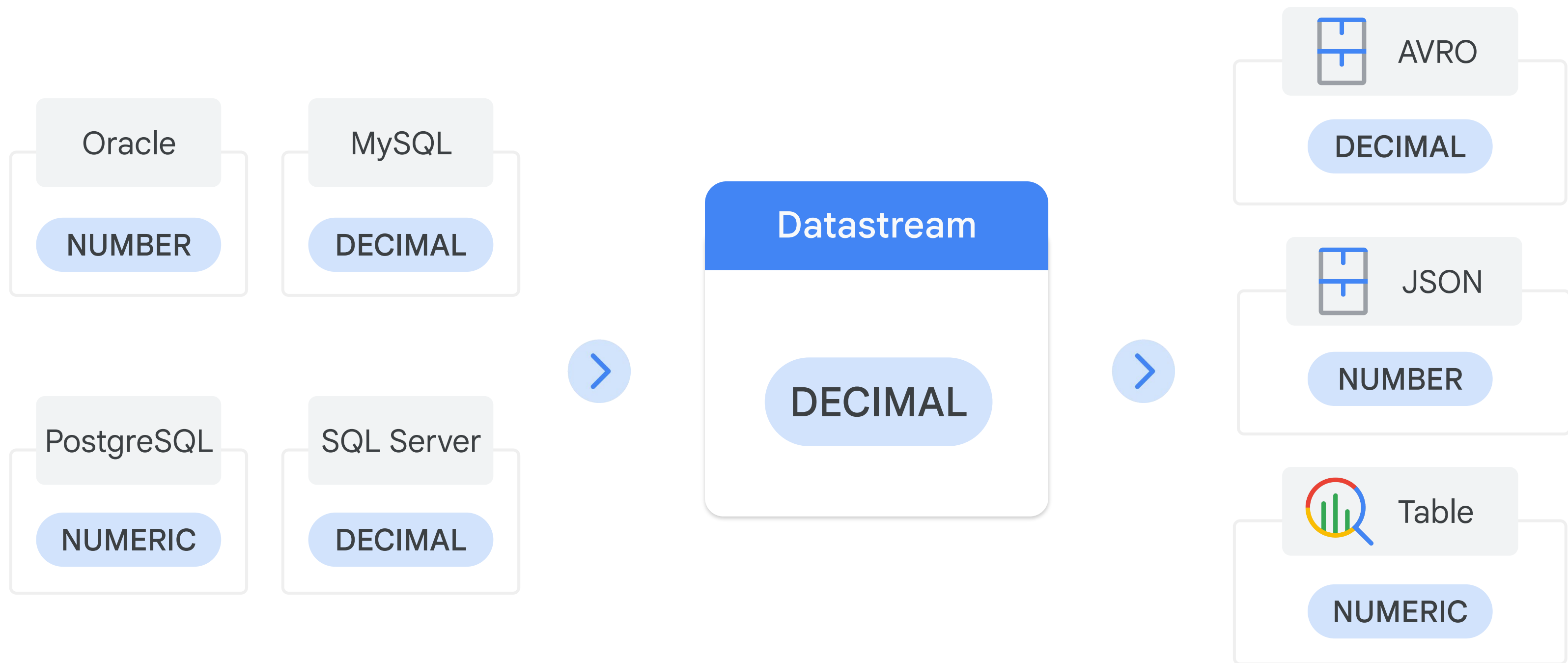
Log-based CDC — the answer for your M&O team

Datastream reads the write-ahead log / transaction log the database is already writing.

- No triggers— nothing is added to your tables or to the write path
- No polling— Datastream never runs change-detection queries against production
- No batch window— there is no 1 AM–4 AM slot where the source gets hammered
- The source keeps serving its workload— capture reads a log, not the tables
- Deletes arrive too— `change_type` and `is_deleted` metadata on every replicated row

Pre-empt the objection before M&O raises it: replication adds no triggers, no polling, no production load.

Datastream uses unified types to map source to destination data types



Compare migration and replication options

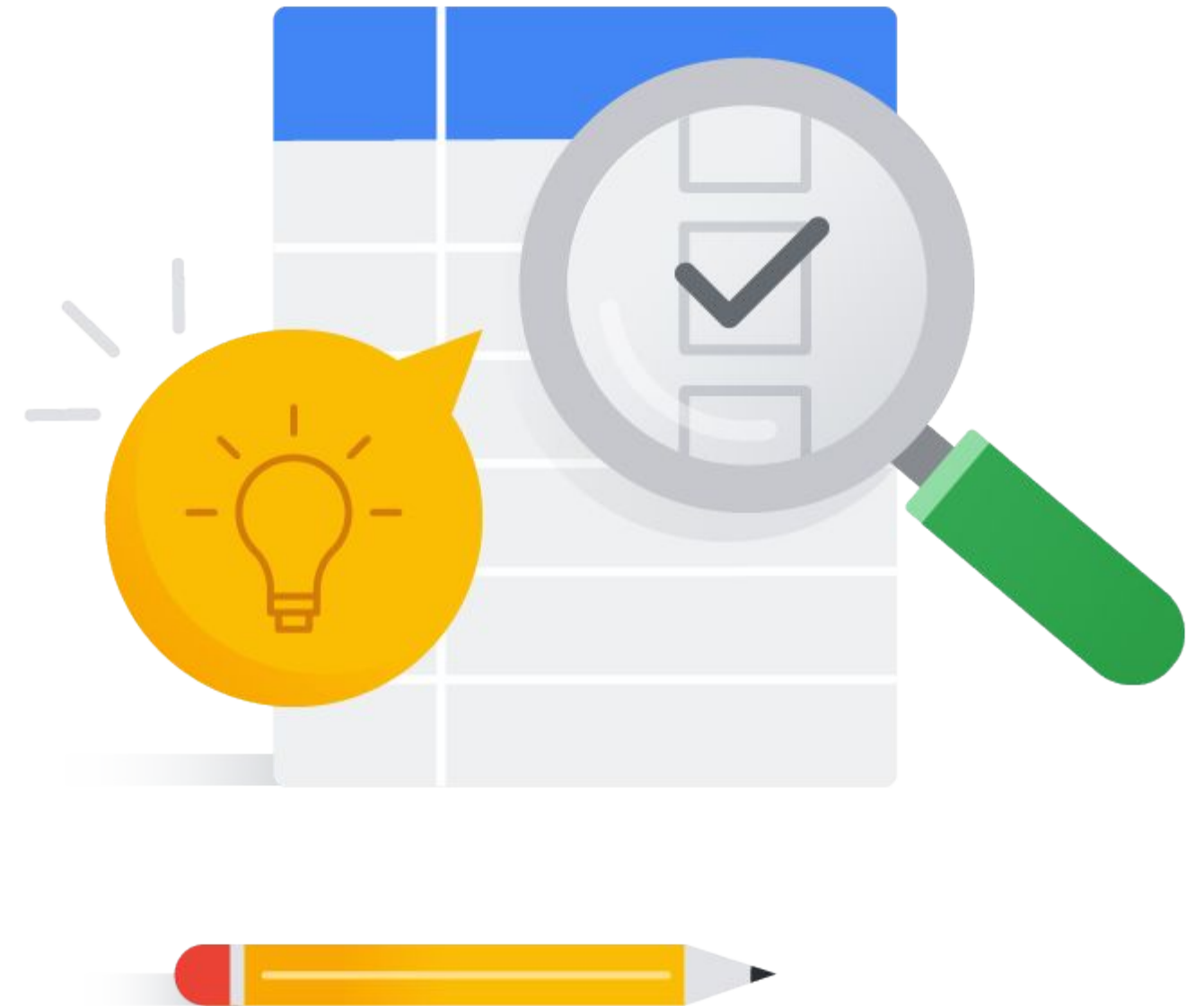
	gcloud storage	Storage Transfer Service	Transfer Appliance	Datastream
Transfer type	online	online	offline	online
Source locations	On-premises, Google Cloud	On-premises, Google Cloud, multicloud	On-premises	On-premises, Google Cloud, multicloud
Data range	less than 1 TB recommended	more than 1 TB recommended	7 TB, 40 TB, 300 TB	10,000 tables per stream
Velocity	Batch	Batch (hourly at minimum)	Batch	Batch and Stream
Data format	any	any	any	structured

Lab (optional): Datastream: PostgreSQL Replication to BigQuery

🕒 45 min

Learning objectives

- Prepare a Cloud SQL for PostgreSQL instance using the Google Cloud console.
- Import data into the Cloud SQL instance.
- Create a Datastream connection profile for the PostgreSQL database.
- Create a Datastream connection profile for the BigQuery destination.
- Create a Datastream stream and start replication.
- Validate that the existing data and changes are replicated correctly into BigQuery.



Lab part 1 — Datastream: PostgreSQL to BigQuery (setup & start)

10 minutes — 11:50–12:00, then lunch while it runs · Google Skills classroom — your pre-warmed lab project

1. Open the module 2 lab — the Cloud SQL for PostgreSQL instance is pre-warmed
2. Follow the instructor click-for-click: create the connection profile for the PostgreSQL source
3. Create the connection profile for the BigQuery destination — regions must match
4. Create the stream and start replication — hands up when stream status shows Running
5. Leave everything running and break for lunch — do not pause or clean up

Why it matters — the stream backfills and replicates while we eat — at 13:00 you validate what landed. PostgreSQL is here because it is free-tier friendly; the mechanics are identical for SQL Server.

Lab part 2 — validate backfill and changes (after lunch)

15 minutes — 13:00–13:15 · BigQuery console + your lab's Cloud SQL instance

1. Open the BigQuery tables the stream created — destination tables appear as the stream runs
2. Validate the backfill: row counts vs the source, eyeball the data
3. Inspect the Datastream metadata columns — `change_type` and `is_deleted` on every row
4. Apply the lab's source change (or your own UPDATE/INSERT) against Cloud SQL
5. Poll the BigQuery table until the change lands — expect roughly a minute

Why it matters — one-time backfill, then incremental changes forever — this is the end of nightly truncate-and-reload for your largest tables.

Demo — CDC validation: a change lands while we watch

Inside lab part 2 — 13:00–13:15 · Instructor screen — Cloud SQL source to BigQuery destination

1. Run an UPDATE and an INSERT against the source Cloud SQL instance
2. Watch the Datastream stream status and throughput tick in the console
3. Poll the BigQuery destination table until the changed rows appear
4. Read the metadata on the new rows — `change_type` tells insert from update
5. Note the latency: source commit to BigQuery in about a minute, no job ran

Why it matters — your data changed while we ate lunch — no nightly job, no extract window, no SSIS package ran. That is the whole reason you are migrating.

M2 — What you now have

- ✓ The distinction that frames your project— one-time backfill, then a continuous change stream, in one operation
- ✓ SQL Server confirmed as a supported Datastream source— your migration path is first-class
- ✓ The M&O answer in your pocket— log-based CDC: no triggers, no polling, no production load
- ✓ A stream you started before lunch and validated after— backfill plus a live change, seen end to end
- ✓ Next: data that still arrives as files— the Extract & Load pattern (M3, 13:15)